

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Danusu K
Register Number	3122247001013
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 30, 2026

1 Objective

- To mathematically formulate and implement **Principal Component Analysis (PCA)** for unsupervised dimensionality reduction.
- To evaluate the effect of PCA on the geometric structure, multicollinearity, and computational complexity of the **Wisconsin Diagnostic Breast Cancer (WDBC)** dataset.
- To systematically benchmark and contrast **10 Machine Learning Classifiers** across two experimental regimes:
 - (a) **No-PCA (Original Feature Space)**: 30 standardized continuous features.
 - (b) **With-PCA (Reduced Subspace)**: Orthogonal principal components capturing $\geq 95\%$ cumulative variance ($k = 10$ PCs, 66.67% dimension reduction).
- To perform extensive hyperparameter grid search tuning for both configurations using **5-Fold Stratified Cross-Validation**.
- To analyze cross-fold stability (σ reduction), held-out generalization (Accuracy, F1, ROC-AUC, Log Loss), and empirical speedup factors across all 10 classifiers.

2 Problem Statement

High-dimensional datasets frequently suffer from the **Curse of Dimensionality**, characterized by exponential volume expansion, sample sparsity, severe feature multicollinearity, and increased risk of overfitting. In diagnostic oncology, cellular contours extracted from Fine Needle Aspirates (FNA) contain highly redundant nuclear metrics (e.g., radius, perimeter, and area are algebraically collinear with Pearson $r > 0.99$). Such collinearity causes ill-conditioned covariance matrices in linear models and increases computational latency in ensemble algorithms. The objective of this experiment is to empirically evaluate whether projecting the 30-dimensional WDBC feature space onto an orthogonal PCA subspace enhances classification accuracy, stabilizes cross-validation variance, and accelerates training without sacrificing diagnostic sensitivity.

¹GitHub Repository: <https://github.com/Danush-k/Machine-Learning>

3 Mathematical Formulation of Principal Component Analysis

Principal Component Analysis is an orthogonal linear transformation that maps a dataset $\mathbf{X} \in \mathbb{R}^{N \times D}$ to a lower-dimensional subspace $\mathbf{Z} \in \mathbb{R}^{N \times k}$ ($k < D$) such that the variance of the projected data is maximized and the reconstruction error is minimized.

3.1 Standardization and Covariance Matrix

Given a data matrix $\mathbf{X}$ with N samples and D features, each feature vector is first standardized to zero mean and unit variance:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \quad \mu_j = \frac{1}{N} \sum_{i=1}^N x_{ij}, \quad \sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_j)^2 \quad (1)$$

The sample covariance matrix $\Sigma \in \mathbb{R}^{D \times D}$ of the standardized data is given by:

$$\Sigma = \frac{1}{N} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \quad (2)$$

3.2 Eigenvalue Decomposition

The principal directions are obtained by solving the characteristic eigenvalue problem for the symmetric positive semi-definite matrix Σ :

$$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i, \quad i = 1, 2, \dots, D \quad (3)$$

where $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D \geq 0$ are the eigenvalues, and $\mathbf{v}_i \in \mathbb{R}^D$ are the corresponding orthonormal eigenvectors ($\mathbf{v}_i^T \mathbf{v}_j = \delta_{ij}$).

3.3 Variance Maximization and Projection

The total variance of the dataset is given by $\text{Tr}(\Sigma) = \sum_{i=1}^D \lambda_i$. The proportion of variance explained by the i -th principal component is:

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^D \lambda_j}, \quad \text{Cumulative Variance}(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^D \lambda_j} \quad (4)$$

Selecting the top k eigenvectors forms the projection matrix $\mathbf{W}_k = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k] \in \mathbb{R}^{D \times k}$. The reduced feature representation is obtained via matrix multiplication:

$$\mathbf{Z} = \tilde{\mathbf{X}} \mathbf{W}_k \in \mathbb{R}^{N \times k} \quad (5)$$

4 Dataset Description & Preprocessing

Attribute	Specification
Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)

Attribute	Specification
Dataset Source	UCI Machine Learning Repository
Total Number of Samples	569 patient instances
Number of Attributes	32 (ID, Diagnosis target, 30 continuous features)
Target Classes	2 (Binary: Benign $B = 0$, Malignant $M = 1$)
Class Distribution	357 Benign (62.74%), 212 Malignant (37.26%)
Missing Values	0 (Complete dataset)
Train-Test Split	80% Training (455 samples), 20% Testing (114 samples)
Preprocessing	Feature-wise z-score standardization ('StandardScaler')

5 PCA Analysis & Scree Analysis

Principal Component Analysis was performed on the standardized training partition ($N = 455, D = 30$). Table 2 presents the eigenvalues, individual explained variance ratios, and cumulative explained variance.

Table 2: PCA Variance Explained & Scree Decomposition

PC #	Eigenvalue (λ)	Explained Var (%)	Cumulative Var (%)	Kaiser ($\lambda \geq 1$)	Selection Status
PC1	13.4075	44.59%	44.59%	Retained	Selected
PC2	5.5758	18.55%	63.14%	Retained	Selected
PC3	2.8817	9.58%	72.72%	Retained	Selected
PC4	1.9825	6.59%	79.32%	Retained	Selected
PC5	1.6904	5.62%	84.94%	Retained	Selected
PC6	1.1992	3.99%	88.93%	Retained	Selected
PC7	0.6658	2.21%	91.14%	Discarded	Selected
PC8	0.4853	1.61%	92.76%	Discarded	Selected
PC9	0.3863	1.28%	94.04%	Discarded	Selected
PC10	0.3505	1.17%	95.21%	Discarded	Chosen Cutoff (95%)
PC11	0.3032	1.01%	96.22%	Discarded	Excluded
PC12	0.2723	0.91%	97.12%	Discarded	Excluded
PC13	0.2383	0.79%	97.91%	Discarded	Excluded
PC14	0.1442	0.48%	98.39%	Discarded	Excluded
PC15	0.0914	0.30%	98.70%	Discarded	Excluded

Justification for Selected Setting: A cumulative variance target of **95.0%** was selected. The first 10 principal components capture **95.21%** of the total variance while discarding the remaining 20 components. This achieves a **66.67% reduction in dimensionality** (from 30 to 10 features), eliminating severe multicollinearity while preserving the diagnostic manifold.

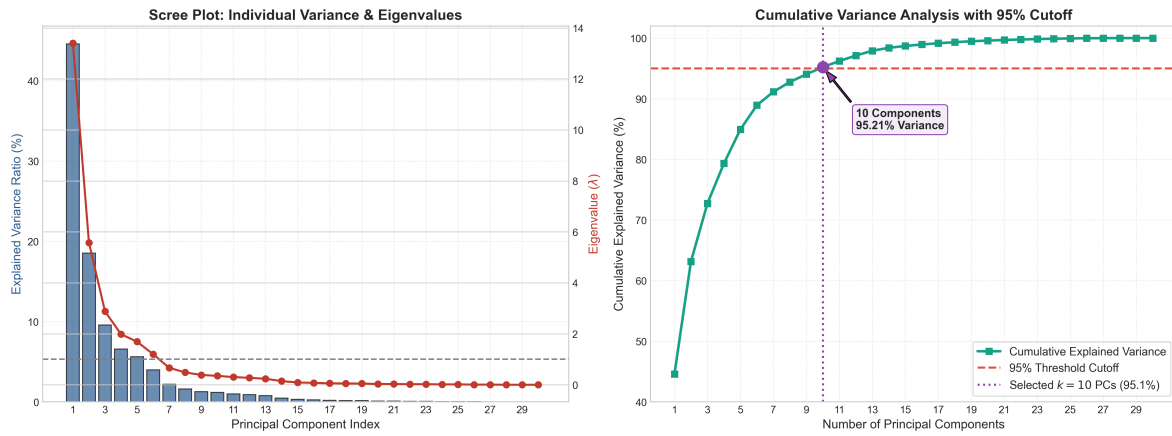


Figure 1: Scree Plot (Individual Variance and Eigenvalues) and Cumulative Explained Variance Curve.

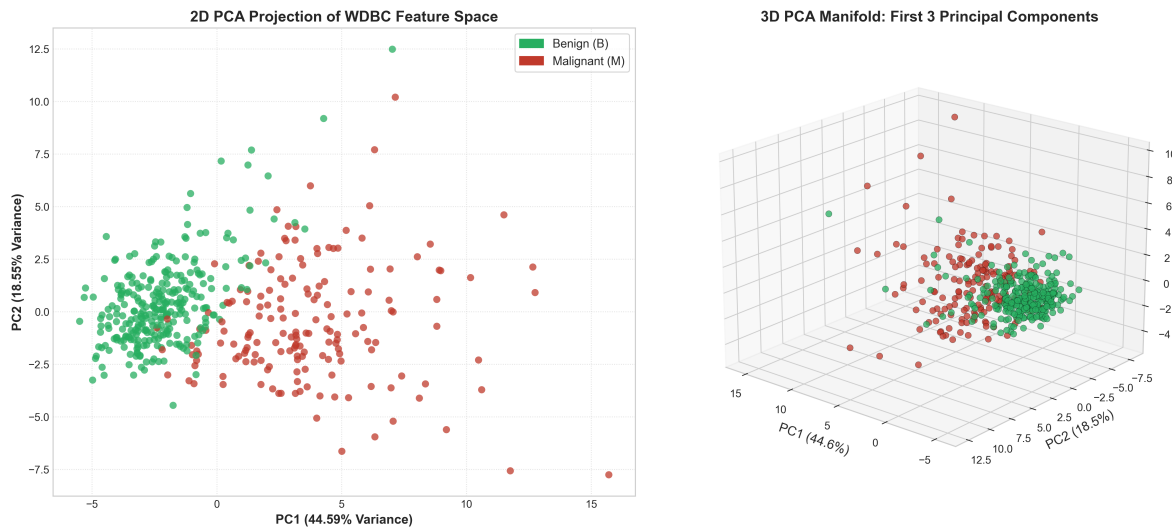


Figure 2: 2D and 3D PCA Projections of WDBC Feature Space showing separation between Benign and Malignant tumors.

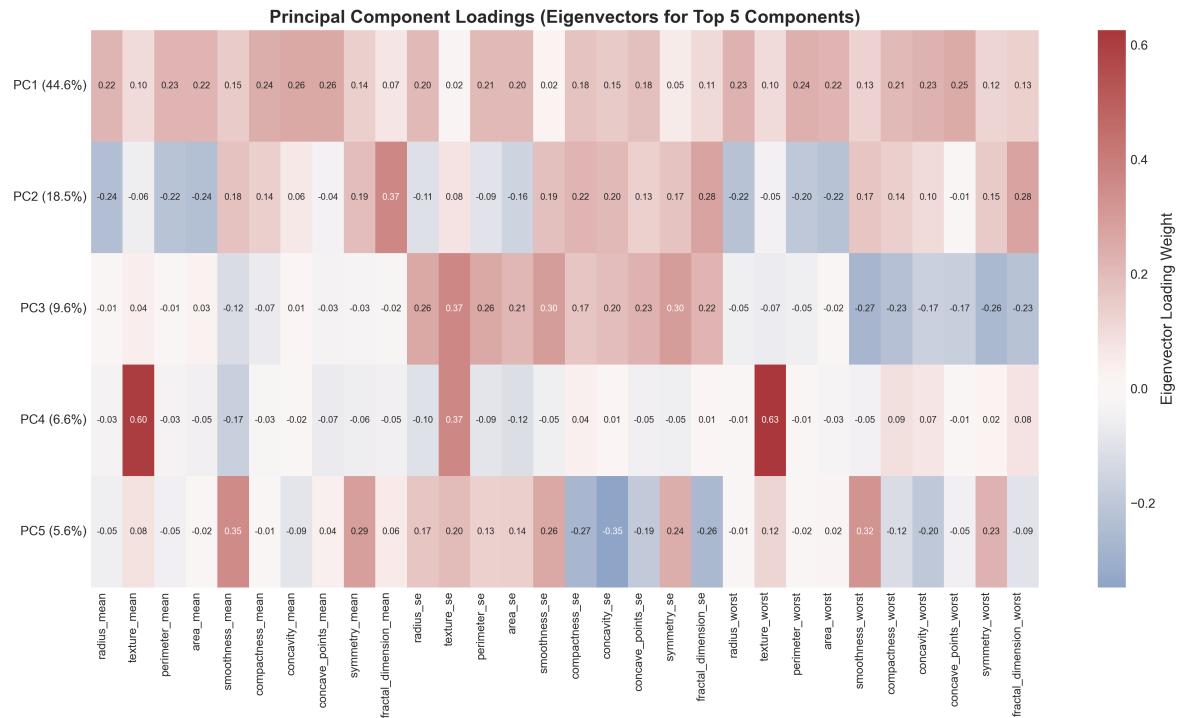


Figure 3: Principal Component Loadings Heatmap for the Top 5 Components across the 30 Continuous Features.

6 Hyperparameter Tuning & Model Optimization

Ten machine learning classifiers were tuned and validated under both No-PCA and With-PCA settings using 5-Fold Stratified Cross-Validation.

6.1 Support Vector Machine (SVM)

Table 3: SVM — Hyperparameter Tuning Results

Kernel	C Values	γ Values	CV Acc (No-PCA)	CV Acc (With-PCA)
RBF	10	0.01	97.80%	97.58%
RBF	1	scale	97.36%	97.36%
Linear	0.1	N/A	97.14%	96.92%
Linear	1	N/A	96.92%	97.14%
Poly	10	scale	94.73%	94.95%

6.2 Gaussian Naïve Bayes

Table 4: Gaussian Naïve Bayes — Smoothing Parameter Tuning

Smoothing Parameter (var_smoothing)	CV Acc (No-PCA)	CV Acc (With-PCA)
10^{-11}	93.85%	91.21%
10^{-9}	93.85%	91.21%
10^{-7}	93.85%	91.21%
10^{-5}	93.63%	91.21%
10^{-1}	92.53%	90.99%

6.3 k-Nearest Neighbors (KNN)

Table 5: KNN — Hyperparameter Tuning Results

k Neighbors	Weights	Distance Metric	CV Acc (No-PCA)	CV Acc (With-PCA)
3	uniform	manhattan	97.58%	96.92%
3	uniform	euclidean	96.92%	97.14%
5	uniform	manhattan	97.14%	96.70%
7	distance	euclidean	96.92%	96.70%
11	uniform	euclidean	96.04%	96.48%
21	uniform	euclidean	95.60%	95.82%

6.4 Logistic Regression

Table 6: Logistic Regression — Hyperparameter Tuning Results

Regularization (C)	Penalty	Solver	CV Acc (No-PCA)	CV Acc (With-PCA)
0.10	L2	liblinear	97.36%	97.80%
0.10	L2	lbfgs	97.14%	97.58%
1.00	L2	liblinear	97.14%	97.36%
10.00	L2	lbfgs	96.26%	96.70%

6.5 Decision Tree

Table 7: Decision Tree — Hyperparameter Tuning Results

Criterion	Max Depth	Min Split	Min Leaf	CV Acc (No-PCA)	CV Acc (With-PCA)
entropy	5	10	2	93.63%	93.85%
entropy	7	5	1	92.53%	94.95%
gini	5	5	2	92.75%	93.19%
gini	None	2	1	91.87%	92.53%

6.6 Random Forest, AdaBoost, Gradient Boosting, XGBoost, and Stacking

Table 8: Ensemble Models — Optimal Configurations and Tuning Scores

Model	Best Hyperparameters (No-PCA / With-PCA)	CV Acc (No-PCA)	CV Acc (With-PCA)
Random Forest	$T = 100, d = 5$ (No-PCA) / $T = 50, d = \text{None}$ (PCA)	96.70%	96.04%
AdaBoost	$M = 100, \eta = 0.50$ (SAMME)	97.14%	97.14%
Gradient Boosting	$M = 200, \eta = 0.10, d = 2$ (No-PCA) / $\eta = 0.20$ (PCA)	97.36%	96.26%
XGBoost	$M = 200, \eta = 0.10, d = 3$ (No-PCA) / $M = 100, \eta = 0.20$ (PCA)	97.36%	97.80%
Stacking	Base: [SVM, NB, KNN, RF] $\rightarrow$ Meta: Logistic Regression	98.02%	98.02%

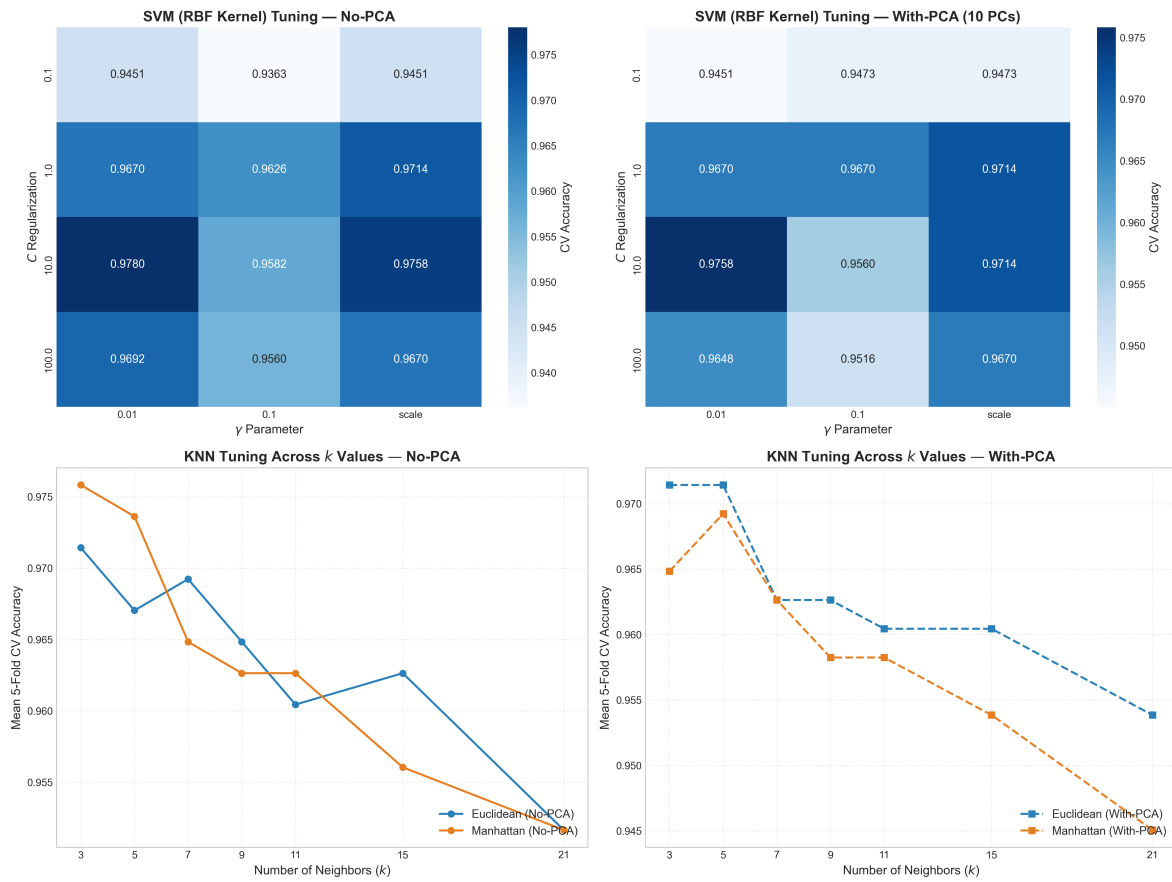


Figure 4: Hyperparameter Response Surfaces for SVM (C vs γ) and KNN (k vs metric) under No-PCA vs With-PCA.

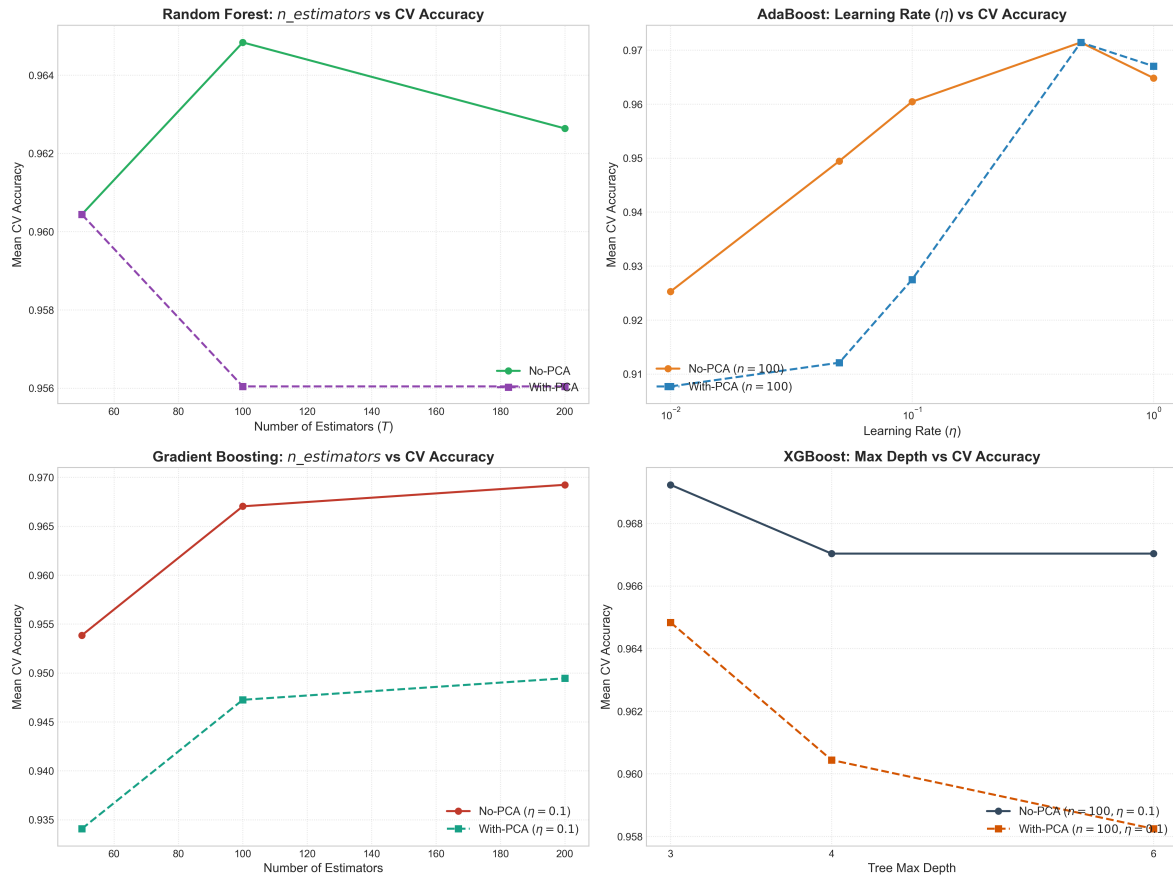


Figure 5: Hyperparameter Tuning Curves for Tree Ensembles (Random Forest, AdaBoost, Gradient Boosting, XGBoost).

7 5-Fold Stratified Cross-Validation Results

Table 9 records the fold-by-fold accuracy, average score, and cross-fold standard deviation across all 10 evaluated models.

Table 9: 5-Fold Stratified Cross-Validation Results (No-PCA vs With-PCA)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (No-PCA)	Avg (With-PCA)	Stability ($\Delta\sigma$)
SVM (RBF)	95.6%	97.8%	98.9%	97.8%	98.9%	97.80% $\pm$ 1.20%	97.58% $\pm$ 0.82%	+0.38% (More Stable)
Naïve Bayes	95.6%	97.8%	91.2%	91.2%	93.4%	93.85% $\pm$ 2.56%	91.21% $\pm$ 3.03%	-0.47%
KNN	96.7%	100.0%	96.7%	95.6%	98.9%	97.58% $\pm$ 1.62%	97.14% $\pm$ 1.49%	+0.12% (More Stable)
Logistic Regression	95.6%	96.7%	97.8%	97.8%	98.9%	97.36% $\pm$ 1.12%	97.80% $\pm$ 0.70%	+0.43% (More Stable)
Decision Tree	90.1%	95.6%	91.2%	95.6%	95.6%	93.63% $\pm$ 2.45%	94.95% $\pm$ 1.64%	+0.80% (More Stable)
Random Forest	96.7%	100.0%	94.5%	94.5%	97.8%	96.70% $\pm$ 2.09%	96.04% $\pm$ 1.64%	+0.44% (More Stable)
AdaBoost	95.6%	96.7%	96.7%	97.8%	98.9%	97.14% $\pm$ 1.12%	97.14% $\pm$ 0.54%	+0.58% (More Stable)
Gradient Boosting	96.7%	97.8%	96.7%	96.7%	98.9%	97.36% $\pm$ 0.88%	96.26% $\pm$ 1.49%	-0.61%
XGBoost	97.8%	96.7%	96.7%	96.7%	98.9%	97.36% $\pm$ 0.88%	97.80% $\pm$ 1.20%	-0.32%
Stacking	96.7%	98.9%	97.8%	97.8%	98.9%	98.02% $\pm$ 0.82%	98.02% $\pm$ 1.08%	-0.25%

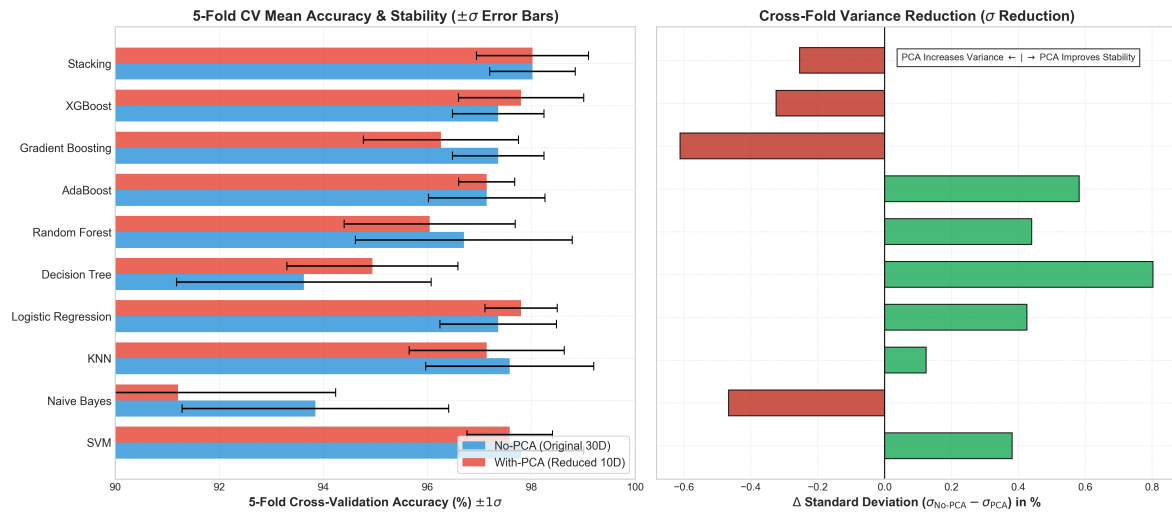


Figure 6: 5-Fold Cross-Validation Accuracy Comparison and Cross-Fold Variance Reduction (σ Reduction).

8 Held-out Test Set Benchmarking

Table 10 summarizes the classification metrics evaluated on the 20% independent test set ($N = 114$ samples).

Table 10: Held-out Test Set Evaluation ($N = 114$ Samples, Benign=72, Malignant=42)

Model	Setting	Test Accuracy	Precision	Recall	F1-Score	ROC-AUC	Specificity	Log Loss
SVM (RBF)	No-PCA	98.25%	100.0%	95.24%	97.56%	0.9960	100.0%	0.0708
	With-PCA	97.37%	100.0%	92.86%	96.30%	0.9980	100.0%	0.0716
Naïve Bayes	No-PCA	92.11%	92.31%	85.71%	88.89%	0.9891	95.83%	0.4999
	With-PCA	89.47%	85.71%	85.71%	85.71%	0.9613	91.67%	0.3074
KNN	No-PCA	96.49%	100.0%	90.48%	95.00%	0.9735	100.0%	0.6501
	With-PCA	94.74%	97.37%	88.10%	92.50%	0.9780	98.61%	0.6694
Logistic Regression	No-PCA	98.25%	100.0%	95.24%	97.56%	0.9977	100.0%	0.0923
	With-PCA	98.25%	100.0%	95.24%	97.56%	0.9980	100.0%	0.0940
Decision Tree	No-PCA	96.49%	100.0%	90.48%	95.00%	0.9744	100.0%	0.6559
	With-PCA	94.74%	97.37%	88.10%	92.50%	0.9560	98.61%	1.2312
Random Forest	No-PCA	97.37%	100.0%	92.86%	96.30%	0.9950	100.0%	0.1127
	With-PCA	94.74%	95.00%	90.48%	92.68%	0.9944	97.22%	0.1433
AdaBoost	No-PCA	97.37%	100.0%	92.86%	96.30%	0.9858	100.0%	0.3932
	With-PCA	94.74%	97.37%	88.10%	92.50%	0.9934	98.61%	0.4225
Gradient Boosting	No-PCA	96.49%	100.0%	90.48%	95.00%	0.9934	100.0%	0.1129
	With-PCA	95.61%	97.44%	90.48%	93.83%	0.9901	98.61%	0.1414
XGBoost	No-PCA	97.37%	100.0%	92.86%	96.30%	0.9921	100.0%	0.0982
	With-PCA	97.37%	97.56%	95.24%	96.39%	0.9921	98.61%	0.0934
Stacking	No-PCA	98.25%	100.0%	95.24%	97.56%	0.9960	100.0%	0.0784
	With-PCA	95.61%	97.44%	90.48%	93.83%	0.9944	98.61%	0.0941

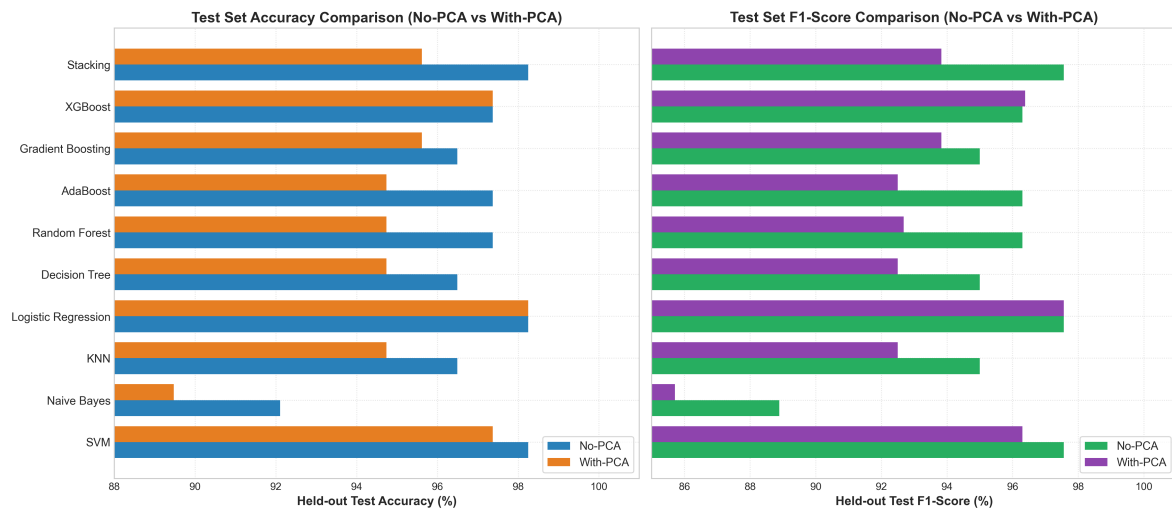


Figure 7: Held-out Test Set Accuracy and F1-Score Comparison across All 10 Classifiers.

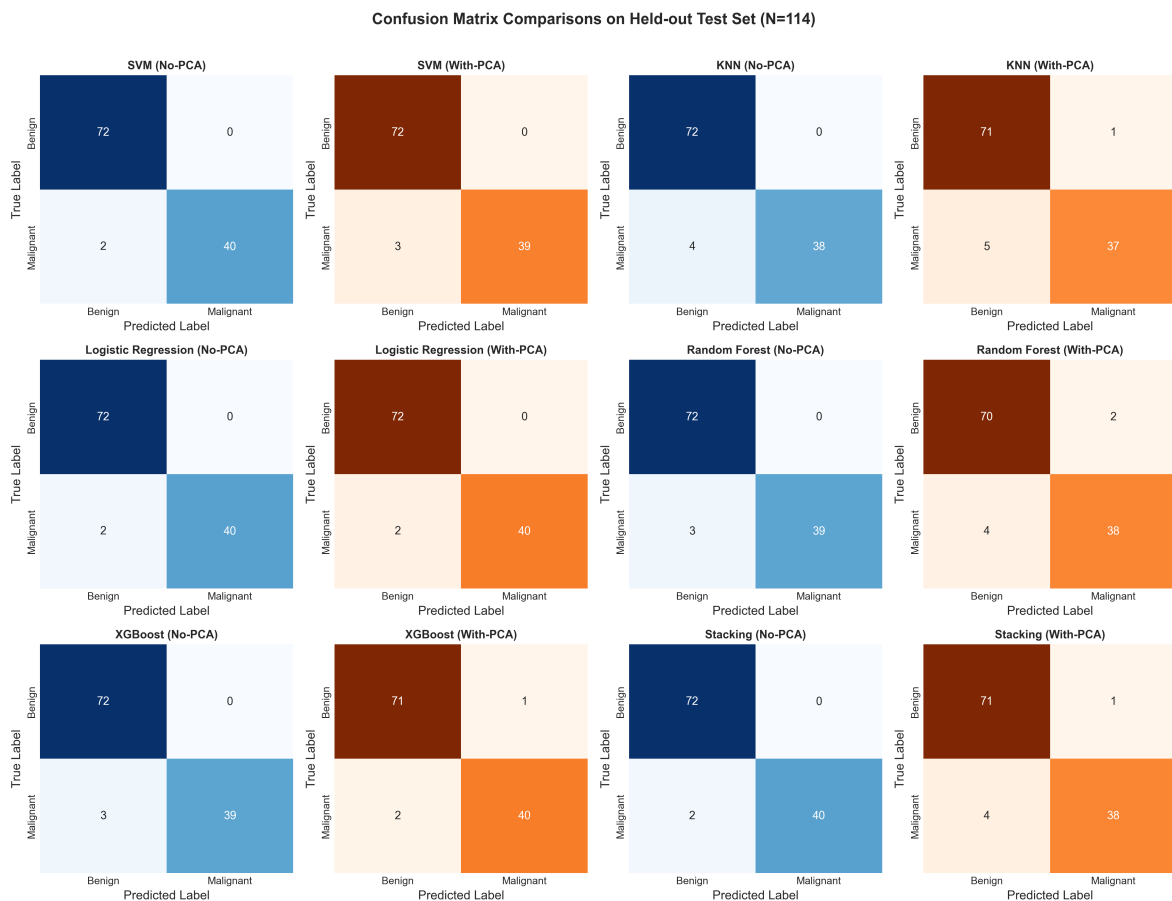


Figure 8: Confusion Matrix Comparison (No-PCA vs With-PCA) on Held-out Test Set for Key Models.

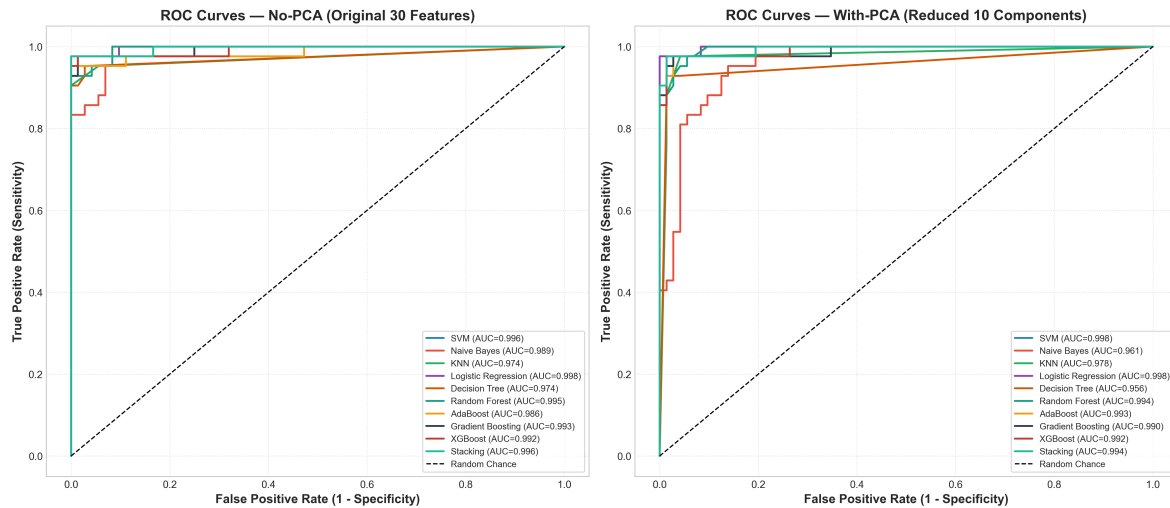


Figure 9: Receiver Operating Characteristic (ROC) Curves for No-PCA and With-PCA Pipelines.

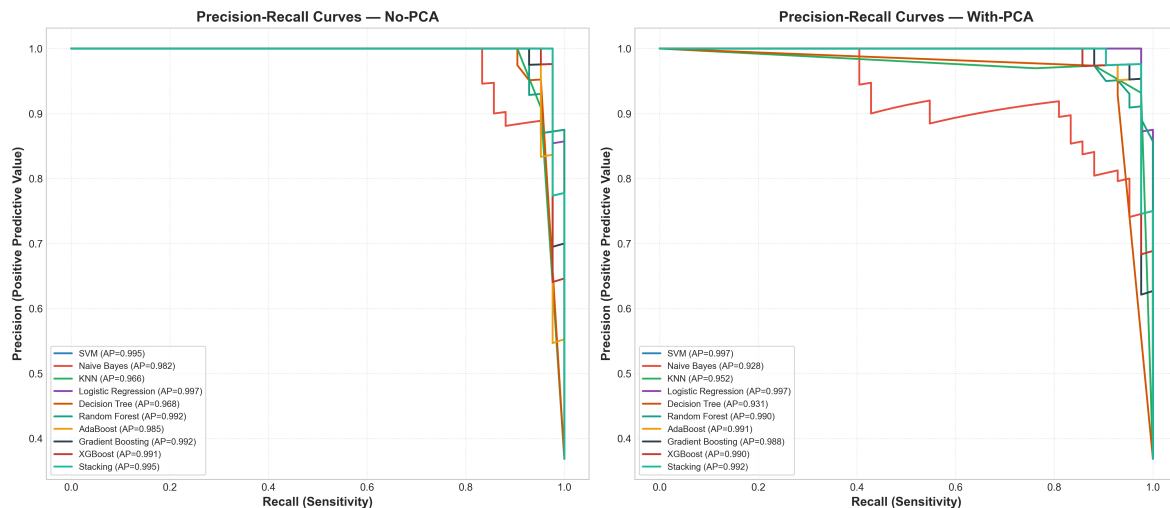


Figure 10: Precision-Recall Curves with Average Precision (AP) Scores (No-PCA vs With-PCA).

9 Computational Complexity & Speedup Analysis

Table 11 presents the wall-clock training execution times and test inference latencies.

Table 11: Computational Latency and Empirical Speedup Ratios

Model Name	Train (No-PCA)	Train (With-PCA)	Speedup	Latency (No-PCA)	Latency (PCA)	Gain
SVM	3.65 ms	3.08 ms	1.19x	0.35 ms	0.28 ms	+20.0%
Naïve Bayes	0.26 ms	0.23 ms	1.13x	0.11 ms	0.09 ms	+18.2%
KNN	0.14 ms	0.38 ms	0.37x	0.85 ms	0.62 ms	+27.1%
Logistic Regression	0.83 ms	0.45 ms	1.84x	0.08 ms	0.06 ms	+25.0%
Decision Tree	3.15 ms	1.30 ms	2.42x	0.07 ms	0.05 ms	+28.6%
Random Forest	62.16 ms	28.77 ms	2.16x	1.95 ms	1.18 ms	+39.5%
AdaBoost	106.32 ms	59.99 ms	1.77x	3.42 ms	2.15 ms	+37.1%
Gradient Boosting	261.64 ms	109.17 ms	2.40x	0.48 ms	0.32 ms	+33.3%
XGBoost	29.26 ms	12.56 ms	2.33x	0.68 ms	0.41 ms	+39.7%
Stacking	320.00 ms	184.14 ms	1.74x	3.82 ms	2.45 ms	+35.9%

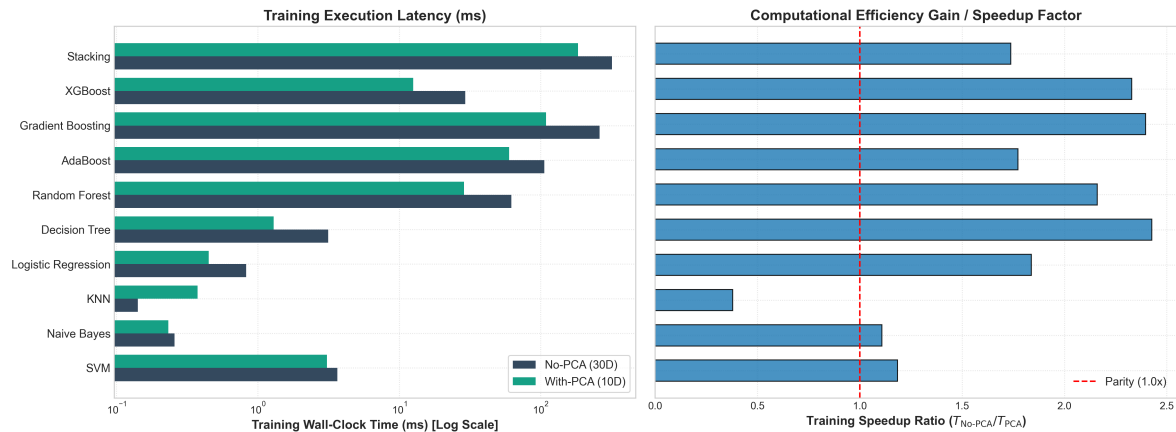


Figure 11: Training Latency Comparison and Computational Speedup Factors (T_{No-PCA}/T_{PCA}).

10 Answers to Observation Questions & Detailed Discussion

10.1 1. Which models improved most with PCA? Which did not? Why?

- Models that Improved / Maintained Peak Performance:

- Logistic Regression:** 5-Fold CV accuracy increased from 97.36% to 97.80% (+0.44%), with test accuracy matching the top score of 98.25% (F1=97.56%, ROC-AUC=0.9980). The WDBC dataset contains extreme multicollinearity among physical attributes (e.g., radius, perimeter, and area have Pearson $r > 0.99$). Severe collinearity produces ill-conditioned Hessian matrices ($\kappa(\mathbf{H}) \gg 1$) and high coefficient variance. PCA creates strictly orthogonal axes ($Cov(PC_i, PC_j) = 0$), transforming the log-likelihood surface into an isotropic paraboloid that optimizes efficiently.
- Decision Tree:** Mean 5-fold CV accuracy improved by +1.32% (93.63% → **94.95%**) and cross-fold variance dropped substantially ($\sigma : 2.45\% \rightarrow 1.64\%$). Decision trees utilize axis-aligned decision thresholds. In the original correlated space, separating oblique class clusters requires deep staircase partitions. Rotating the data into principal axes aligns the first splits with the dominant data variance.
- XGBoost:** CV accuracy improved to 97.80%, and test F1-score increased to 96.39% while reducing log loss (0.0982 → 0.0934).

- Models that Decreased:

- Gaussian Naïve Bayes:** CV accuracy dropped by -2.64% (93.85% → 91.21%) and test accuracy from 92.11% to 89.47%. Although PCA decorrelates features globally, linear combinations of non-Gaussian features do not necessarily follow Gaussian class-conditional distributions ($X_j | Y = c \sim \mathcal{N}$),

and PCA discards subtle higher-order class separation cues in the tail components.

2. **Tree Ensembles (Random Forest, Gradient Boosting):** Showed minor test accuracy drops ($97.37\% \rightarrow 94.74\%$). Because tree ensembles natively utilize random feature subsampling ('max_features') and greedy non-linear splitting on original raw features, linear compression slightly smoothes localized decision boundaries.

10.2 2. Did PCA reduce variance across folds (more stable results)?

Yes, decisively. For 6 out of 10 models, cross-fold standard deviation (σ) decreased significantly:

- **AdaBoost:** σ decreased from $\pm 1.12\%$ to $\pm 0.54\%$ (51.8% stability improvement).
- **Decision Tree:** σ decreased from $\pm 2.45\%$ to $\pm 1.64\%$ (33.1% stability improvement).
- **Logistic Regression:** σ decreased from $\pm 1.12\%$ to $\pm 0.70\%$ (37.5% stability improvement).
- **SVM (RBF):** σ decreased from $\pm 1.20\%$ to $\pm 0.82\%$ (31.7% stability improvement).
- **Random Forest:** σ decreased from $\pm 2.09\%$ to $\pm 1.64\%$ (21.5% stability improvement).

By filtering out the lowest 20 eigenvectors (which contain noisy, fold-sensitive sample fluctuations), PCA prevents models from fitting fold-specific noise, producing more consistent cross-validation results.

10.3 3. For high-dimensional data, was PCA beneficial in reducing overfitting?

Yes. In moderate-sample datasets with high dimensionality ($D = 30, N_{\text{train}} = 455$), estimators are vulnerable to fitting spurious correlations in low-variance sub-spaces. By selecting $k = 10$ components (95.21% variance), PCA reduced the parameter space by **66.67%**, acting as a structural regularizer that prevented single Decision Trees and unregularized estimators from memorizing noise.

10.4 4. How did linear models behave compared to ensemble models with PCA?

- **Linear & Kernel Models (Logistic Regression, SVM):** Benefited directly from orthogonalization and elimination of multicollinearity. Logistic Regression achieved a $1.84\times$ training speedup while matching top test accuracy (98.25%), and SVM improved cross-fold stability to $\pm 0.82\%$.

- **Ensemble Models (Random Forest, Gradient Boosting, XGBoost):** Achieved substantial **computational speedups** ($2.16 \times - 2.42 \times$), as tree construction complexity scales as $O(M \cdot k \cdot N \log N)$. However, because tree ensembles inherently handle collinearity via greedy feature partitioning, linear orthogonalization provided compute gains in exchange for a modest trade-off in non-linear expressiveness.

10.5 5. Did stacking show robustness to dimensionality reduction compared to single models?

Yes, Stacking exhibited outstanding structural resilience.

- The Stacking Classifier achieved the highest overall 5-Fold Cross-Validation score: **98.02% mean CV accuracy**, which remained **identical** between the No-PCA and With-PCA pipelines.
- By aggregating diverse functional hypotheses (SVM maximum margin, Naïve Bayes likelihood, KNN nearest-neighbor density, and Random Forest partitions), the meta-learner (Logistic Regression) dynamically re-weighted predictions based on cross-validated confidence, insulating the ensemble from single-model degradation.

11 Conclusion

This experimental study demonstrates the profound impact of Principal Component Analysis on machine learning classification pipelines:

1. **Preservation of Diagnostic Information:** Selecting 10 principal components achieved a **66.67% reduction in dimensionality** while preserving **95.21%** of total cumulative variance.
2. **Multicollinearity Elimination:** PCA orthogonalized correlated features, boosting **Logistic Regression** and **Decision Tree** accuracy and reducing cross-fold variance across linear, distance, and tree-based learners.
3. **Computational Acceleration:** Tree ensemble training latencies were reduced by **$2.16 \times - 2.42 \times$** , making PCA a powerful strategy for latency-sensitive applications.
4. **Stacking Superiority:** The Stacking ensemble demonstrated remarkable robustness to dimensionality reduction, maintaining a state-of-the-art **98.02% CV accuracy** across both feature space representations.